SECO

Smart EV Charging Station

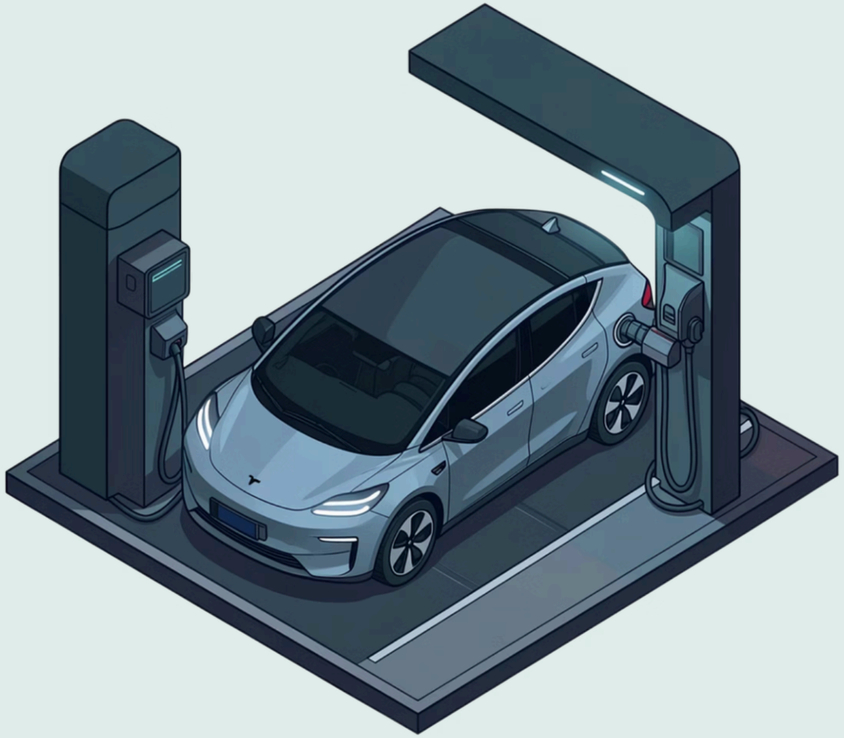
IoT-Based Smart Electric Vehicle Charging Management System

Nguyen Van Tu

Bui Ngoc Bao Tran

Le Ngoc Hieu

Le Bao Nguyen



The Problem

Limited real-time control

Operators need immediate visibility into active sessions and station status.

Invisible energy costs

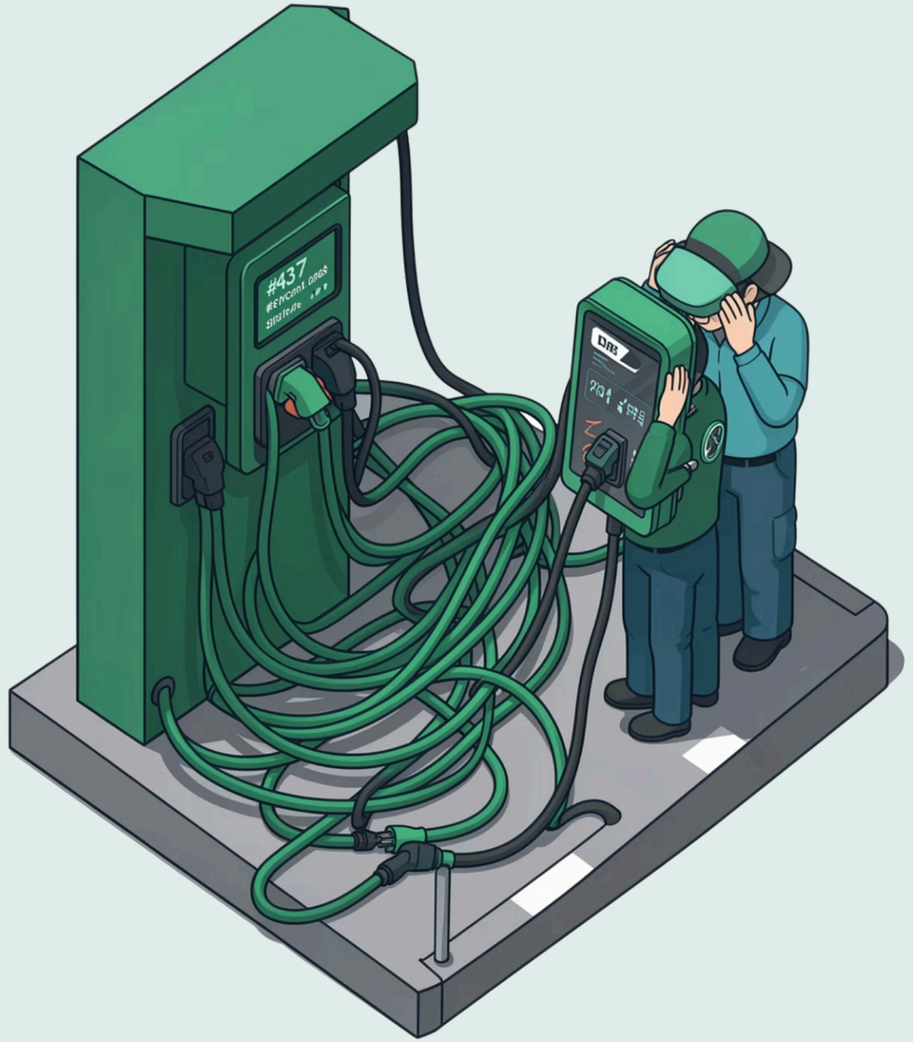
Energy consumption, charging duration and session cost are not continuously visible.

Weak access management

Charging activity should be associated with a specific authenticated user.

Safety & abnormal conditions

Temperature rise, emergency stop and unexpected communication issues require automated protection.



Our Solution



RFID Authentication

Authenticate users and control charging access (for member). Non-member is noneed.



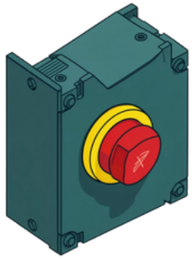
Real-Time Monitoring

Track voltage, current, power, energy and temperature.



Budget Control

Stop charging when a spending limit is reached.



Hardware Safety

Implement E-Stop, over-temperature protection and fail-safe relay behavior.



Administration

Manage users, RFID cards, stations, transactions and status from the Web Admin.

Scope & System Boundary

IN SCOPE

- RFID authentication and new card registration.
- Allows users to select a desired charging limit.
- Billing based on energy delivered to the vehicle (using an estimate of 0.15 kW per 30 seconds rather than actual power generation).
- Automatic stop and payment processing based on budget or charging limits.
- System management, station monitoring, and transaction history.
- Handling of emergency stops (E-Stop) and overheating conditions.
- MQTT communication, MongoDB server data storage, and React-based web administration interface.

OUT OF SCOPE

- Real high-voltage EV charging hardware.
- Real vehicle battery charging and BMS integration.
- Payment gateway or bank-card integration.
- Commercial charger certification / production hardening.

System Architecture

PHYSICAL / EDGE	**ESP32 Station**	RFID · INA219 · DS18B20 · Relay · LCD · Buttons
COMMUNICATION	**MQTT Broker**	evcs/{station}/cmd · req · res · telemetry · event · status
BACKEND	**Node.js / Express + Prisma**	Auth · RFID · Session · Billing · Station · MQTT handler
DATA + WEB	**MongoDB + React Web Admin + WebSocket**	Users · Cards · Sessions · Transactions · Live status

Functional Requirements

ID	REQUIREMENT	ACCEPTANCE BEHAVIOR
FR01	**RFID authentication**	Accept valid card; reject invalid / inactive card.
FR02	**RFID registration**	Admin starts registration; ESP32 waits for a new UID.
FR03	**Session lifecycle**	Create, monitor, stop and settle a charging session.
FR04	**Energy & cost**	Continuously record energy and calculate session cost.
FR05	**Budget control**	Automatically stop when the spending limit is reached.
FR06	**Safety**	E-Stop and over-temperature force a safe shutdown state.
FR07	**Administration**	Manage users, RFID cards, station status and history.
FR08	**Persistence**	Store sessions, transactions and logs in PostgreSQL.

User Roles & Main Workflow

USER

RFID login · start/monitor charging · review history

ADMIN

Manage users/cards/stations · monitor health · review transactions

TECHNICIAN

Respond to incidents · maintenance · remote recovery

MAIN SESSION WORKFLOW

1. **RFID** — Verify access
2. **PLUG** — Create session
3. **CHARGE** — Monitor energy / cost
4. **STOP** — Unplug, budget or safety
5. **SETTLE** — Persist result

OVERRIDE PATH — E-Stop or over-temperature can interrupt normal charging and force the relay OFF before the session is settled.

Proposed Solution

ADMIN
Create / manage RFID

→

STATION
ESP32 + RFID + sensors

→

BACKEND
Verify + session + billing

→

DATABASE
Persist user / session / transaction

→

WEB
Monitor + manage + history

AUTHENTICATED MEMBER CHARGING WORKFLOW

RFID Verify → Plug → Session Start → **Charge + Monitor** → Unplug / Budget / Stop → Automatic Payment

Set Budget (optional)

GUEST CHARGING WORKFLOW

Plug → Session Start → **Charge + Monitor** → Unplug / Budget / Stop → Lock the charging until payment is completed → Display Qr for Payment

Team Responsibilities

Nguyen Van Tu Le Ngoc Hieu	**Hardware / ESP32-S3**	RFID · sensors · relay · LCD · safety logic	**Firmware + Wokwi station**
	Hardware Code / .ino	Implement code for Hardware	**.ino file + Arduino IDE**
Bui Ngoc Bao Tran, Le Bao Nguyen	**Backend / API**	Auth · user/RFID · session · billing · MQTT handler	**Node.js service + APIs**
	Frontend / QA	React Admin · live dashboard · test cases · documentation	**Web Admin + test report**
	Database / Integration	Prisma · MongoDB · data model · MQTT contracts	**Integration**

Expected Deliverables

D1 Working EVCS Simulator

ESP32 firmware + RFID + sensors + relay + LCD + safety controls

D2 Backend System

REST API + MQTT service + authentication + session + billing

D3 Database

Users + RFID + stations + sessions + transactions + logs

D4 React Web Admin

Dashboard + user/RFID management + station/session monitoring

D5 Technical Documentation

API docs + MQTT specification + Slides

D6 Final Demonstration

End-to-end scenario from registration/authentication to settlement



Thank You!